Verified-Execution Harness

Architecture & Design Reference for an OpenCode Agent Wrapper

How to design the controllers, contracts, and files behind a harness that lets an autonomous coding agent (OpenCode) act on a real workspace, while every mutation is sandboxed, snapshotted, independently re-verified, and only ever accepted through a single, auditable gate decision.

Scope	Single-repository, single-agent harness; multi-agent extensions noted where relevant
Audience	The engineer building the harness itself
Companion artifacts	harness/ reference package, test_all.py, test_more.py (all executed while writing this document)
Status	Design reference -- pair with your own threat model before production use

Table of Contents

1. Executive Summary

2. Design Principles

3. System Overview

- 3.1 The two questions the harness answers
- 3.2 End-to-end pipeline

4. Harness Entry & Session Model

5. Context Controller

6. Model / Profile Controller

7. Orchestration Controller

8. The OpenCode Agent Boundary

9. Permission Controller

10. Execution Controller

11. Workspace Snapshot & Diff

12. Verification Runtime

- 12.1 Trusted catalog
- 12.2 Identity validation
- 12.3 Check selection
- 12.4 Running checks & aggregation

13. Gate Runtime

- 13.1 Verification vs. gate
- 13.2 Reconciliation
- 13.3 Decision table

14. Evaluation Layer

15. Observability, Evidence & Receipts

16. Contracts & Schemas

17. Security Model

- 17.1 Symlink defense
- 17.2 TOCTOU defense
- 17.3 Process containment
- 17.4 Trust boundaries

18. File & Module Layout

19. Testing Strategy

20. Failure Modes & Operational Notes

Appendix A: Verified Code Listings

Appendix B: Glossary

1. Executive Summary

OpenCode is a capable but unsupervised actor: given a task, it decides on its own which shell commands to run, which files to edit, and which MCP tools to call. The harness described in this document does not make OpenCode smarter or safer by itself -- it wraps OpenCode in a set of independent controllers that (a) constrain what it is allowed to touch, (b) record exactly what happened, and (c) refuse to call a run successful until the change has been checked by something OpenCode does not control.

The central design move is to separate four questions that are easy to accidentally collapse into one:

- **Did the agent's process exit cleanly?** (execution result)
- **Was anything on disk actually touched?** (workspace mutation)
- **Does an independent, trusted check confirm the change is correct?** (verification)
- **Given all of the above, may the harness report success?** (the gate decision)

Each question is answered by a different controller, using a different kind of evidence, and the final decision is a pure function of that evidence -- never of the agent's own claims about what it did. This document specifies the controllers, their contracts, the files they live in, and the security properties each one is responsible for.

> **NOTE.** Every code listing in this document was extracted from a reference package (`harness/`) that was written and executed while producing this document -- including the symlink, TOCTOU, zombie-process, and tamper-detection tests. Treat it as a working skeleton to adapt, not as a spec to re-implement blind.

2. Design Principles

2.1 Never trust the agent's self-report

OpenCode's own claim that "tests pass" or "the change is done" is evidence about the agent's intent, not about the world. The harness only believes things it can independently reproduce: a workspace snapshot it took itself, a check it launched itself with a pinned executable path, an exit code it read from a process it owns.

2.2 Verification and the gate are different layers

A verification runtime answers "what happened when we ran the check?" A gate runtime answers "given that, and everything else we know (risk, permissions, policy, budget), may we proceed?" Conflating them produces systems where a passing test silently overrides a permission violation. Keeping them separate means the gate can refuse to pass a green test suite, and that is a feature.

2.3 Fail closed, not open

Every controller in this design defaults to the safer outcome when it cannot decide: unknown tool -> DENY; missing trusted check -> ASK or BLOCK depending on policy, never silent ALLOW; catalog fails to parse -> abort, don't run a partial catalog; verification mutates the workspace -> re-verify, don't assume the first result still holds.

2.4 Evidence is structured, hashed, and append-only

Nothing in the pipeline is trusted because a controller "says so" in a log line. Every fact that matters to the final decision is captured as an *Evidence* record, fingerprinted with a domain-separated SHA-256 hash, folded into a *receipt*, and the trace of events leading to it is hash-chained so a tampered trace is detectable, not just implausible.

2.5 Contracts are schemas, not conventions

Every JSON object that crosses a controller boundary (trace events, catalogs, receipts, policies) is validated against an explicit schema that rejects both missing *and* unexpected keys. An unexpected key is not harmless metadata -- it is the shape of a future confused-deputy bug.

2.6 Small, replaceable controllers over one large loop

Each controller in Section 3 has one job, a narrow interface, and can be tested in isolation without spinning up OpenCode. This is what makes the reference package in Appendix A runnable and testable on its own -- every module in it has a test that exercises it with real processes and a real filesystem, not mocks.

3. System Overview

3.1 The two questions the harness answers

Strip away the controllers and the whole system is answering two questions in sequence: **"What did the agent actually change?"** and **"Can we prove that change is acceptable?"** Everything upstream (context, model routing, orchestration) exists to produce a good attempt at the task; everything downstream (snapshot, verification, gate, evidence) exists to avoid taking the agent's word for whether that attempt succeeded.

3.2 End-to-end pipeline

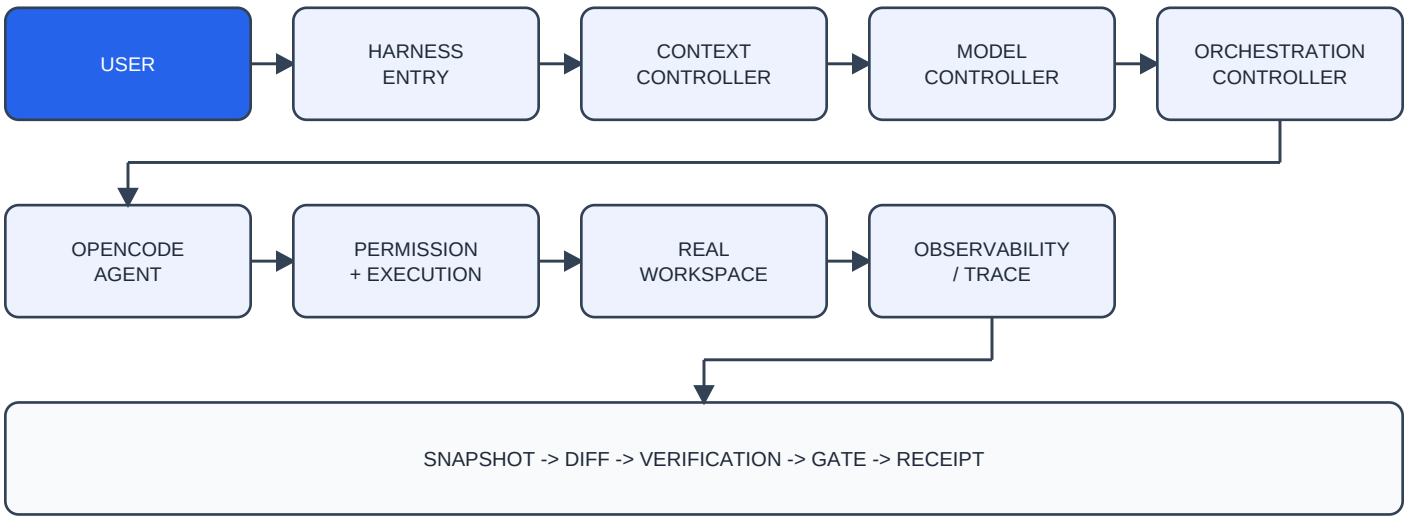


Figure 1. Top-level pipeline. The agent only ever touches the real workspace inside the permission + execution box; everything after it treats that workspace as untrusted data to be re-checked.

Reading the diagram left to right: a user task enters through the harness entry point, which stamps it with identifiers and a budget (Section 4). The context controller assembles a bounded, fingerprinted slice of the repository and task history (Section 5). The model controller picks a profile for the work (Section 6). The orchestration controller turns the task into steps and hands each step to OpenCode (Section 7-8). Every tool call OpenCode makes passes through the permission controller before the execution controller will run it (Sections 9-10). Once OpenCode's process exits, the harness diffs the workspace against a snapshot taken before the run, and only if something actually changed does it proceed to verification and the gate (Sections 11-13).

4. Harness Entry & Session Model

The entry point is the only place identifiers are minted. It receives a task, a workspace path, and a policy, and produces an immutable *RunContext*: a session ID, a job ID, a budget, and the resolved policy the rest of the run must obey. Nothing downstream is allowed to invent its own IDs -- traceability depends on every event carrying the IDs assigned here.

Responsibilities

- Validate the workspace path exists, is a directory, and is owned by the invoking user (reject shared or symlinked workspace roots up front -- see Section 17.1).
- Mint session_id and job_id as safe identifiers (Section 16) and never reuse a job ID across runs.
- Load and validate the acceptance policy and permission rules *before* starting the orchestration controller; a policy that fails schema validation must abort the run, not fall back to a default.
- Construct the initial Budget (token, cost, and attempt ceilings) and the RunMachine state object described in Section 7.
- Open the trace writer (Section 15) as the very first action, so even a crash during context gathering is recorded.

File: harness/entry.py (interface sketch)

```
@dataclass(frozen=True)
class RunContext:
    session_id: str
    job_id: str
    workspace: Path
    policy: AcceptancePolicy
    budget: Budget
    trace: TraceWriter

def start_run(task: TaskSpec, workspace: Path,
              policy_path: Path, catalog_path: Path) -> RunContext:
    """Validates inputs, mints IDs, opens the trace. Raises on any
    malformed policy or catalog -- never starts a run on a fallback."""
```

Sketch only; not part of the tested reference package -- it composes the tested modules from Appendix A.

> **NOTE.** Treat session_id as a trust boundary. If the harness is ever exposed to more than one caller (a CI system and a chat agent, say), the entry point -- not the orchestration controller -- is where you enforce that a session cannot read or resume another session's workspace or budget.

5. Context Controller

The context controller's job is to turn an unbounded amount of candidate context (the whole repo, the whole git history, every prior session) into a bounded, ordered, fingerprinted set of items that fits the model profile's context window. It is a budgeting problem, not a retrieval-quality problem -- retrieval quality belongs in whatever populates the candidate list; this controller's contract is that it never silently exceeds the budget and never silently drops a mandatory item.

Contract

- Every candidate item has a priority (0 is mandatory: the task text, the acceptance criteria) and a pre-computed token estimate.
- Mandatory items that do not fit the budget are a hard error, not a truncation -- the caller must shrink the task or raise the budget.
- Non-mandatory items are dropped or compressed, cheapest and lowest-priority first, via a caller-supplied compress callback so the controller itself stays free of format-specific logic.
- The final selection is fingerprinted so two runs given the same candidate context produce the same fingerprint, and a changed fingerprint always means the context genuinely differs.

```
@dataclass(frozen=True)
class ContextItem:
    key: str                # e.g. "file:src/a.py", "git:diff", "task"
    text: str
    priority: int           # 0 = mandatory, higher = less important
    tokens: int              # pre-computed estimate (integer)

def select_context(items: list[ContextItem], budget_tokens: int,
                   compress=None):
    """Mandatory items first, then by priority. compress(item, room) may
    return a smaller ContextItem (summary) or None to drop the item."""
    chosen, used = [], 0
    ordered = sorted(items, key=lambda i: (i.priority, i.tokens, i.key))
    for it in ordered:
        room = budget_tokens - used
        if it.tokens > room:
            if it.priority == 0:
                raise ContractError(f"mandatory context {it.key} > budget")
            it = compress(it, room) if compress else None
            if it is None or it.tokens > room:
                continue
        chosen.append(it)
        used += it.tokens
    fp = fingerprint([[i.key, fingerprint(i.text, "ctx-item/v1")]
                      for i in chosen], "context/v1")
    return chosen, used, fp
```

harness/context/select.py -- executed against a five-item mixed mandatory/optional budget in test_all.py; the fixture drops the lowest-priority item once the budget is exceeded and returns a stable fingerprint.

6. Model / Profile Controller

This controller answers "which model handles this step," not "what should the model do." It is a router over a small, static set of profiles (core / deep / orchestration in the reference), each with a context window ceiling, a

cost-per-1K-token figure, and a risk ceiling: the highest risk level that profile may be used for unattended.

Routing rule

- A step that needs planning or decomposition routes to the orchestration profile if one exists.
- A step with high complexity or high risk routes to the deep profile.
- Everything else routes to the core profile.
- If the chosen profile's estimated cost would exceed the remaining budget, the controller degrades to the core profile -- but only if the core profile's risk ceiling still covers the task's risk. If even the core profile can't be afforded, that is a hard error, not a silent skip.

```
def route(profiles: dict[str, Profile], t: TaskFeatures) -> Profile:
    if t.needs_planning and "orchestration" in profiles:
        want = "orchestration"
    elif t.complexity >= 2 or t.risk >= 2:
        want = "deep"
    else:
        want = "core"
    p = profiles[want]
    est = p.cost_micro_usd_per_1k * (t.context_tokens // 1000 + 1)
    if t.context_tokens > p.max_context:
        raise ValueError(f"context does not fit profile {p.name}")
    if est > t.remaining_cost_micro_usd:
        core = profiles["core"]          # degrade, never overspend silently
        if core.risk_ceiling < t.risk:
            raise ValueError("cheaper profile not allowed at this risk level")
        if core.cost_micro_usd_per_1k * (t.context_tokens // 1000 + 1) \
                > t.remaining_cost_micro_usd:
            raise ValueError("budget too small for any profile")
        return core
    return p
```

harness/models/route.py -- test_all.py confirms a high-risk, high-complexity task with a tight remaining budget is refused rather than silently downgraded past its risk ceiling.

NOTE. The risk-ceiling check on degradation is the detail worth protecting in code review: a naive "downgrade on low budget" policy would happily run a high-risk step on the cheapest profile once the budget got tight -- exactly when you want more scrutiny, not less.

7. Orchestration Controller

The orchestration controller owns the run's explicit state machine and budget. It decomposes the task into steps, assigns each step a model profile and a tool surface, and decides after every step whether to continue, retry, stop, or escalate to the user. It does not itself call OpenCode's tools -- that is delegated to the agent boundary (Section 8) -- it only decides what should happen next.

Explicit states

The state machine has one property that matters more than its exact shape: illegal transitions raise instead of being silently coerced. A run cannot jump from EXECUTING to TERMINAL_VERIFIED without passing through VERIFYING and GATING, even if a caller's code mistakenly tries to skip them.

```
class RunState(str, Enum):
    CREATED = "created"
    CONTEXT_READY = "context_ready"
    PLANNED = "planned"
    EXECUTING = "executing"
    VERIFYING = "verifying"
    GATING = "gating"
    TERMINAL_VERIFIED = "terminal_verified"
    TERMINAL_BLOCKED = "terminal_blocked"
    TERMINAL_FAILED = "terminal_failed"

S = RunState
_ALLOWED = {
    S.CREATED: {S.CONTEXT_READY, S.TERMINAL_FAILED},
    S.CONTEXT_READY: {S.PLANNED, S.TERMINAL_FAILED},
    S.PLANNED: {S.EXECUTING, S.TERMINAL_FAILED},
    S.EXECUTING: {S.VERIFYING, S.GATING, S.TERMINAL_FAILED},
    S.VERIFYING: {S.GATING, S.TERMINAL_FAILED},
    S.GATING: {S.VERIFYING, S.EXECUTING, S.TERMINAL_VERIFIED,
               S.TERMINAL_BLOCKED, S.TERMINAL_FAILED},
}

class RunMachine:
    def go(self, nxt: RunState) -> None:
        if nxt not in _ALLOWED.get(self.state, set()):
            raise IllegalTransition(f"{self.state.value} -> {nxt.value}")
        self.history.append((self.state, nxt))
        self.state = nxt
```

harness/orchestration/state.py -- test_all.py confirms CONTEXT_READY -> TERMINAL_VERIFIED is rejected as an illegal jump, and that GATING can legally loop back to VERIFYING (this is what a REVERIFY gate outcome, Section 13, drives).

Budget

Budget tracking lives alongside the state machine because both questions -- "is this transition legal" and "can we afford another attempt" -- gate whether orchestration proceeds. Budget.charge() and start_attempt() raise BudgetExceeded the instant a ceiling is crossed, so the orchestration loop never needs to remember to check afterward.

8. The OpenCode Agent Boundary

OpenCode itself -- the model reasoning about the task and deciding on tool calls -- is treated as an untrusted black box. The harness does not need to understand OpenCode's internal reasoning; it only needs a narrow, enforceable seam where every tool call OpenCode wants to make is intercepted before it touches the terminal, the editor, or an MCP server.

The three tool surfaces

Surface	What it can do	Where it is constrained
Terminal	Run a shell command	Permission controller (Section 9) classifies risk; execution controller (Section 10) contains the process
Editor	Create, modify, or delete a workspace file	Permission controller checks the target path; workspace snapshot (Section 11) records the effect regardless of the agent's report
MCP	Call a connected tool/server	Treated as high risk by default (Section 9); any state it changes outside the workspace is invisible to the snapshot and must be evaluated separately

NOTE. MCP calls are the one surface the workspace snapshot cannot see. If a task's MCP tools can mutate state outside the repository (a ticket tracker, a deployment), that mutation needs its own evidence type feeding the gate -- don't assume "no workspace diff" means "nothing happened."

Boundary contract

- Every tool call is wrapped in a single ToolRequest value (Section 9) before it reaches the permission controller -- OpenCode never gets a raw shell handle.
- The boundary does not retry or reinterpret a DENY; it reports the denial back to OpenCode as the tool's result and lets the model decide what to do next within the remaining budget.
- Every tool call and its outcome (allowed / asked / denied, exit code if run) is written to the trace before the boundary returns control to OpenCode, so the trace reflects reality even if OpenCode crashes immediately afterward.

9. Permission Controller

The permission controller answers exactly one question per tool call: "is this action allowed?" It has three possible answers -- ALLOW, ASK, DENY -- and a hard rule that a request matching no rule is denied. There is no fourth, implicit "allow by default" path.

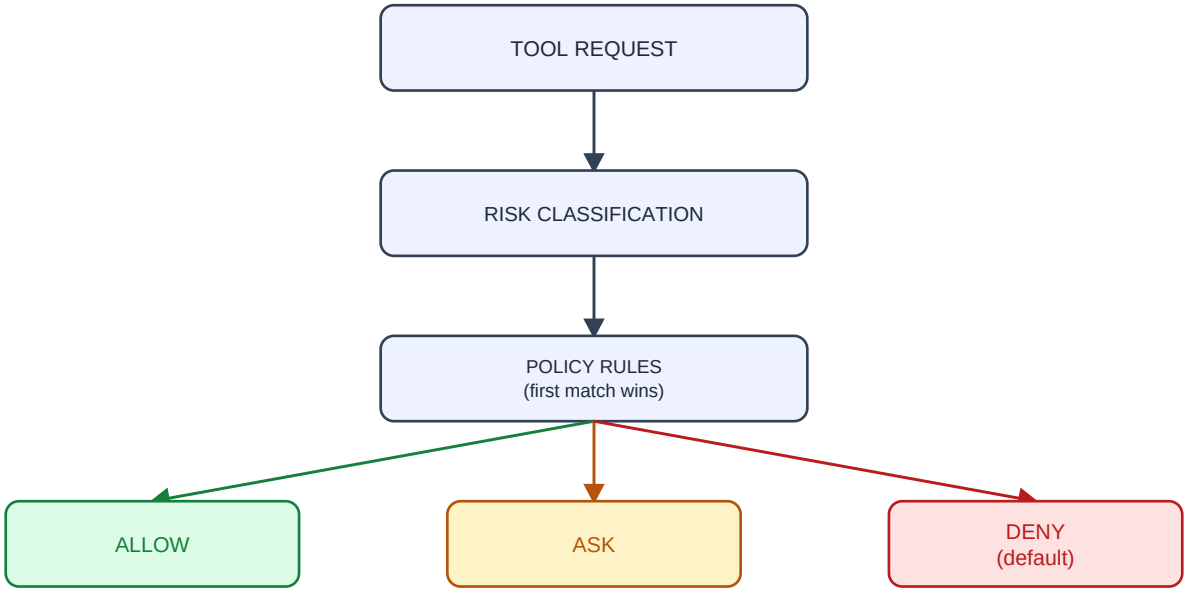


Figure 2. Permission evaluation. Risk classification informs which rules are even relevant, but the verdict always comes from the first matching rule; no match means DENY.

Risk classification

Risk classification is a separate, simpler function from rule evaluation: it looks at the tool surface and, for shell commands, the executable's basename against a fixed high-risk set (sudo, rm, dd, curl, ssh, and similar), plus specific dangerous subcommands like git push or git reset. Editor writes to .git/ or the harness's own metadata directory are always high risk regardless of file extension.

```
_HIGH = {"sudo", "su", "rm", "dd", "mkfs", "chmod", "chown", "curl",
         "wget", "ssh", "scp", "nc", "kill", "pkill"}

def classify_risk(req: ToolRequest) -> str:
    if req.tool == "shell" and req.argv:
        base = os.path.basename(req.argv[0])
        if base in _HIGH:
            return "high"
        if base == "git" and any(a in req.argv for a in ("push", "reset")):
            return "high"
        return "medium"
    if req.tool == "edit":
        p = req.path
        if p.startswith(".git") or p.startswith(".harness"):
            return "high"
        return "low"
    return "high" if req.tool.startswith("mcp:") else "medium"

def evaluate(rules: tuple[Rule, ...], req: ToolRequest) -> Verdict:
    """First match wins. No match -> DENY (default deny)."""
    target = os.path.basename(req.argv[0]) if req.argv else req.path
    for r in rules:
        if fnmatchcase(req.tool, r.tool) and fnmatchcase(target, r.pattern):
            return r.verdict
    return Verdict.DENY
```

harness/permission/policy.py -- test_all.py checks pytest is allowed, rm is denied by falling through to the default, and an edit rule scoped to src/ allows an in-scope path.*

SECURITY. Rule order is the whole security model here: a broad early rule can shadow a narrow later one. Keep the policy file's rules ordered from most specific to least specific, and treat any change to that ordering as a reviewable, tested change -- not a config tweak.

10. Execution Controller

Once a shell command is ALLOWED, the execution controller is what actually runs it, and it is the layer responsible for making sure a misbehaving or hung process cannot take down the harness or outlive the run. Four properties matter: no shell interpolation, resource limits applied before exec, its own process group so the whole tree can be killed at once, and bounded, captured output.

```
def run_contained(argv, *, cwd: str, env: dict[str, str],
                  limits: RunLimits = RunLimits()) -> RunResult:
    if not os.path.isabs(argv[0]):
        raise ValueError("argv[0] must be an absolute path")
    with tempfile.TemporaryFile() as out, tempfile.TemporaryFile() as err:
        proc = subprocess.Popen(
            list(argv), cwd=cwd, env=env, shell=False,   # never a shell
            stdin=subprocess.DEVNULL, stdout=out, stderr=err,
            start_new_session=True,                      # own process group
            preexec_fn=_limits_fn(limits), close_fds=True)
        try:
            proc.wait(timeout=limits.wall_seconds)
        except subprocess.TimeoutExpired:
            timed_out = True
            _kill_group(proc.pid, signal.SIGTERM)
            try:
                proc.wait(timeout=limits.grace_seconds)
            except subprocess.TimeoutExpired:
                pass
        finally:
            _kill_group(proc.pid, signal.SIGKILL)        # reap strays
            proc.wait()
        ...
```

harness/execution/process.py -- test_all.py runs a real 30-second sleep launched as a detached background child of a shell wrapper; after the timeout kills the process group, the leaked child is confirmed reaped (a zombie awaiting init, not a live stray).

Resource limits applied pre-exec

- RLIMIT_CPU and a wall-clock timeout, enforced independently -- a CPU-bound loop and an I/O-bound hang are different failure modes and need different limits.
- RLIMIT_FSIZE to stop a single runaway write from filling the disk.
- RLIMIT_NPROC to cap fork-bombs.
- RLIMIT_CORE set to zero -- core dumps from an untrusted process are an unnecessary disclosure surface.

SECURITY. shell=False and an absolute-path executable are not stylistic choices. Passing a bare command name lets PATH decide what actually runs, and shell=True reintroduces every shell-injection issue the permission controller's classification was trying to avoid.

11. Workspace Snapshot & Diff

A snapshot is taken immediately before OpenCode runs and again immediately after. The diff between them -- not OpenCode's own report of which files it touched -- is the harness's only source of truth about what changed. This is also what feeds check selection in Section 12.3: only paths that actually changed are eligible for verification scope matching.

```
def take_snapshot(root: Path) -> dict[str, Entry]:
    root_s = os.path.realpath(root)
    out: dict[str, Entry] = {}
    stack = [""]
    while stack:
        rel_dir = stack.pop()
        with os.scandir(os.path.join(root_s, rel_dir)) as it:
            for de in it:
                if not rel_dir and de.name in EXCLUDED_TOP:
                    continue
                rel = f"{rel_dir}/{de.name}" if rel_dir else de.name
                st = de.stat(follow_symlinks=False)   # lstat: never follow
                mode = stat.S_IMODE(st.st_mode)
                if stat.S_ISLNK(st.st_mode):
                    out[rel] = Entry("symlink", mode, 0,
                                     "link:" + os.readlink(de.path))
                elif stat.S_ISDIR(st.st_mode):
                    out[rel] = Entry("dir", mode, 0, "")
                    stack.append(rel)
                elif stat.S_ISREG(st.st_mode):
                    out[rel] = Entry("file", mode, st.st_size,
                                     _hash_file(de.path))
                else:
                    out[rel] = Entry("other", mode, 0, "")
    return out
```

harness/workspace/snapshot.py -- test_all.py mutates a file, adds a directory, deletes a file, plants a symlink pointing at /etc/passwd, and confirms the resulting Diff lists exactly those four changes with the symlink recorded as its target string, never followed and read.

take_snapshot uses lstat (via follow_symlinks=False) and O_NOFOLLOW when hashing file contents, so a symlink is recorded as a symlink -- never silently dereferenced into hashing whatever it happens to point at, inside or outside the workspace.

diff() reduces two snapshots to three sorted path lists (added, modified, deleted); workspace_fingerprint() reduces a whole snapshot to a single hash so a receipt (Section 15) can name "the workspace state this decision is about" in one field.

12. Verification Runtime

Verification is where the harness stops trusting anything OpenCode said and instead runs an independent, pre-registered check against the changed paths. "Independent" is doing a lot of work in that sentence, and Sections 12.1-12.2 are about making that word true in practice, not just in intent.

12.1 Trusted catalog

The catalog is a JSON document, loaded from a path *outside* the workspace, describing every check the harness is allowed to run: its executable, working directory, scope (which changed paths it covers), cost rank, timeout, allowed environment variables, and a set of "immutable inputs" -- config files whose hash is pinned so the agent cannot weaken the check (loosening a lint rule, say) as part of the same run it is trying to get verified.

```
def load_catalog(catalog_path: Path, workspace: Path) -> Catalog:
    real = catalog_path.resolve(strict=True)
    ws = workspace.resolve(strict=True)
    if _is_inside(real, ws):
        raise TrustError("catalog must live OUTSIDE the workspace")
    assert_trusted_ancestry(real)
    raw = real.read_bytes()                 # read ONCE; parse those bytes
    doc = json.loads(raw)
    exact_keys(doc, ["schema_version", "checks"], where="catalog")
    ...
    return Catalog(fingerprint(doc, "catalog/v1"), tuple(checks))

def assert_trusted_ancestry(path: Path) -> None:
    """Every ancestor must be owned by root/us and not group/world writable."""
    me = os.geteuid()
    for d in (path, *path.parents):
        st = d.stat()
        if st.st_mode & 0o022:
            raise TrustError(f"{d} is group/world writable")
        if st.st_uid not in (0, me):
            raise TrustError(f"{d} has untrusted owner {st.st_uid}")
```

harness/verification/catalog.py -- test_all.py confirms a catalog placed under a world-writable directory (a plain /tmp path) is correctly rejected by ancestry validation, and that a catalog path inside the workspace is rejected outright.

> **NOTE.** The catalog is read once into memory and parsed from those exact bytes. Reading it twice (once to hash, once to parse) is a classic TOCTOU opening if the file lives on a filesystem another process can write to between the two reads.

12.2 Identity validation

Loading the catalog establishes trust in the *document*; validate_check establishes trust in the specific check about to run, immediately before it runs, not once at startup. It re-resolves the executable to an absolute, real path, confirms that path is outside the workspace and is a regular, executable file with trusted ancestry, confirms the working directory stays inside the workspace, and re-hashes every immutable input against the pinned digest in the catalog.

```
def validate_check(spec: CheckSpec, workspace: Path) -> str:
    """Identity validation. Returns the command fingerprint or raises."""
    ws = workspace.resolve(strict=True)
    exe = Path(spec.argv[0])
    if not exe.is_absolute():
        raise TrustError(f"{spec.check_id}: executable must be absolute")
    exe = exe.resolve(strict=True)
    if _is_inside(exe, ws):
        raise TrustError(f"{spec.check_id}: executable is inside workspace")
    if not (stat.S_ISREG(exe.stat().st_mode) and os.access(exe, os.X_OK)):
        raise TrustError(f"{spec.check_id}: not an executable file")
    assert_trusted_ancestry(exe)
    cwd = (ws / spec.cwd).resolve(strict=True)
    if not _is_inside(cwd, ws):
        raise TrustError(f"{spec.check_id}: cwd escapes workspace")
    for rel, want in spec.immutable_inputs:
        got = sha256_file(ws / rel)          # config the agent may not weaken
        if got != want:
            raise TrustError(f"{spec.check_id}: {rel} was modified")
    return fingerprint({...}, "command/v1")
```

harness/verification/catalog.py -- test_all.py confirms a check whose executable lives inside the workspace is rejected, and that modifying pytest.ini after the catalog pins its hash correctly fails validate_check on the next attempt.

NOTE. Requiring the executable to be outside the workspace is what stops an agent from writing its own "test runner" that always exits 0 and getting it invoked as a trusted check. Combined with pinned immutable inputs, the agent can change the code under test, but not the thing judging it.

12.3 Check selection

Given the set of changed paths from Section 11 and the catalog's checks, selection answers "which checks apply, and in what order." A check applies if any of its scope globs match any changed path; applicable checks run cheapest (cost_rank) first. Any changed path matched by no check at all is returned separately as uncovered -- this is what lets the gate distinguish "verified" from "verified, except for that one file nothing checks."

```
def covers(check: CheckSpec, path: str) -> bool:
    return any(fnmatchcase(path, g) for g in check.scope)

def select_checks(checks, changed: tuple[str, ...]):
    """Return (ordered checks to run, changed paths nobody covers)."""
    applicable = sorted((c for c in checks
                         if any(covers(c, p) for p in changed)),
                        key=lambda c: (c.cost_rank, c.check_id))
    uncovered = tuple(p for p in changed
                      if not any(covers(c, p) for c in applicable))
    return applicable, uncovered
```

harness/verification/select.py -- test_all.py confirms a two-path change (src/a.py, docs/x.md) against a catalog that only scopes src/ and tests/* selects the src check and reports docs/x.md as uncovered.*

12.4 Running checks & aggregation

Each selected check is launched through the execution controller (Section 10) with only its allow-listed environment variables and its own timeout, and the result is mapped to one of three statuses: PASSED (clean exit), FAILED (non-zero exit or timeout), or UNAVAILABLE (the check itself could not be trusted or launched -- a missing binary, a broken symlink, a failed identity check). UNAVAILABLE is deliberately not the same as PASSED: the harness cannot distinguish "the check would have passed" from "the check never ran," so it must not report success either way.

```
def aggregate(results, uncovered=(), ignorable=()):
    """None means 'no trusted check applied'. Any FAILED wins, then any
    UNAVAILABLE or uncovered path; only full coverage can PASS."""
    left = [p for p in uncovered
            if not any(fnmatchcase(p, g) for g in ignorable)]
    if not results:
        return None
    statuses = {r.status for r in results}
    if VS.FAILED in statuses:
        return VS.FAILED
    if VS.UNAVAILABLE in statuses or left:
        return VS.UNAVAILABLE
    return VS.PASSED
```

harness/verification/runtime.py -- test_more.py exercises a real /bin/true, /bin/false, a killed 5-second sleep under a 1-second timeout, a missing executable, a nonzero exit code, and a check whose executable lives inside the workspace, confirming each maps to the correct status and reason string.

13. Gate Runtime

13.1 Verification vs. gate

Verification answers "what happened when we ran the check?" The gate answers "given that, and everything else we know, may we continue or declare success?" The gate's inputs are therefore wider than verification's: permission violations recorded during execution, the step's risk level, whether the workspace changed at all, the acceptance policy, and whether the post-verification workspace still matches what was verified (Section 13.2).

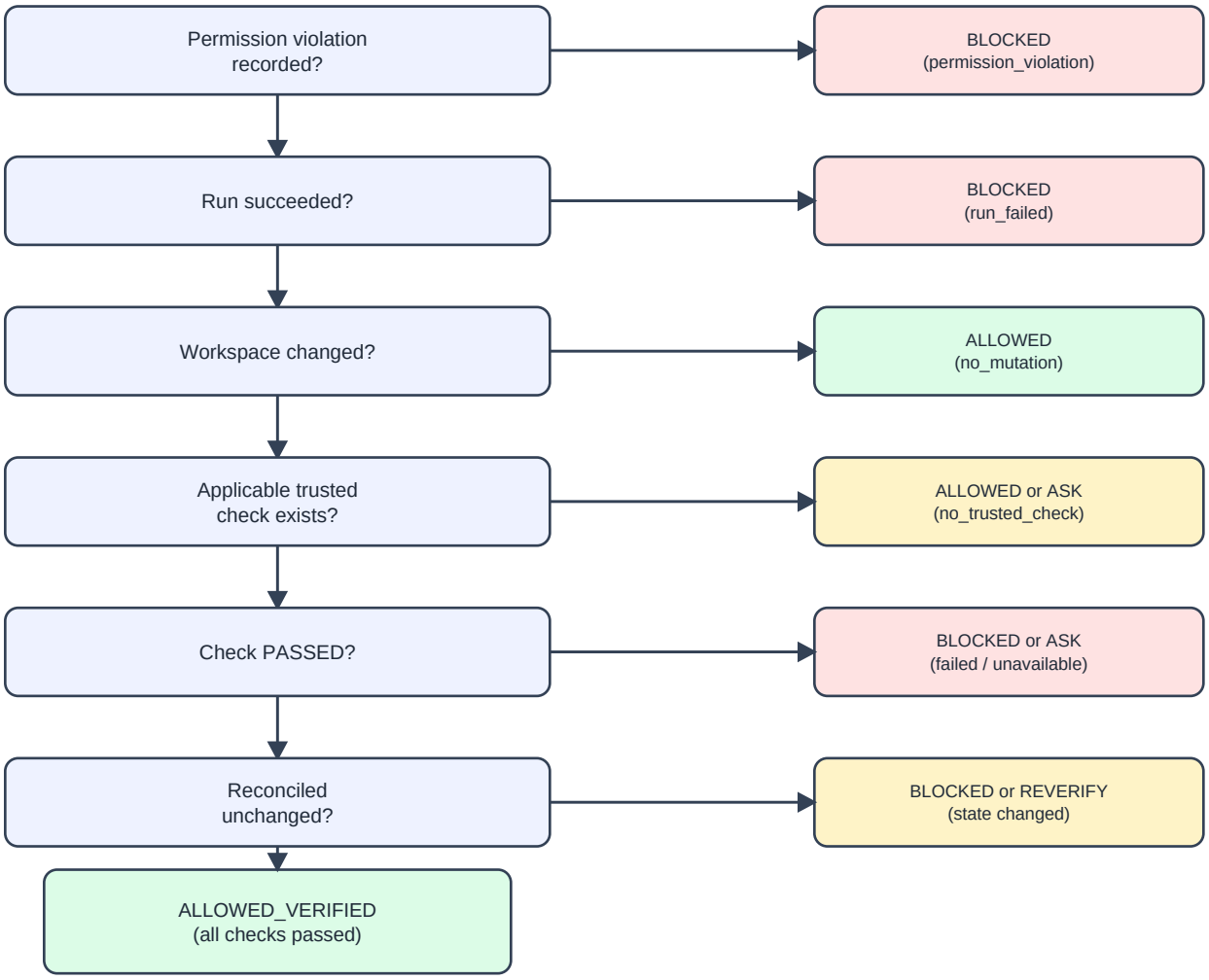


Figure 3. Gate decision tree. A permission violation or a failed run short-circuits everything else; only a change that both passes its trusted check and reconciles to an unchanged state reaches ALLOWED_VERIFIED.

13.2 Reconciliation

A verification check is itself a process running inside the workspace's cwd; nothing stops a check from touching files as a side effect (a coverage report, a cache directory, a lockfile). Reconciliation takes a snapshot immediately before and after running the checks and diffs them the same way Section 11 diffs the agent's own run. If nothing changed, the verification result stands. If something did change, the gate cannot simply trust a result computed against a workspace state that no longer exists -- it must re-verify against the new state, bounded by max_reverify so a flaky check cannot loop forever.

```
@dataclass(frozen=True)
class Reconciliation:
    unchanged: bool
    changed_paths: tuple[str, ...]
    mutation_revision: int
    workspace_fp: str

def reconcile(pre_check, post_check, mutation_revision: int) -> Reconciliation:
    d = diff(pre_check, post_check)
    rev = mutation_revision if not d else mutation_revision + 1
    return Reconciliation(not d, d.changed, rev,
                          workspace_fingerprint(post_check))
```

harness/gate/reconcile.py -- reuses the same Diff/snapshot primitives tested in Section 11, so reconciliation and the initial agent-run diff share one tested code path rather than two.

13.3 Decision table

Condition (checked in order)	State	Decision
Any permission violation recorded	BLOCKED	BLOCKED
Agent process did not exit cleanly	RUN_FAILED	BLOCKED
Workspace unchanged	ALLOWED_NO_MUTATION	ALLOWED
No applicable trusted check exists	ALLOWED_NOT_VERIFIED	ALLOWED if policy permits low-risk unverified, else ASK_USER
Trusted check FAILED	BLOCKED	BLOCKED
Trusted check UNAVAILABLE	UNVERIFIABLE	ASK_USER
Trusted check PASSED, never reconciled	BLOCKED	BLOCKED (missing reconciliation is a programming error, not a policy call)
Trusted check PASSED, reconciliation changed state, retries left	REVERIFY	BLOCKED (loop back to verification)
Trusted check PASSED, reconciliation changed state, retries exhausted	BLOCKED	BLOCKED
Trusted check PASSED and reconciled unchanged	ALLOWED_VERIFIED	ALLOWED

```
def decide(g: GateInput) -> GateOutcome:
    if g.permission_violations:
        return _out(GateState.BLOCKED, B, "permission_violation")
    if not g.run_succeeded:
        return _out(GateState.RUN_FAILED, B, "agent_run_failed")
    if not g.workspace_changed:
        return _out(GateState.ALLOWED_NO_MUTATION, A, "no_mutation")
    if g.verification is None:
        if g.policy.allow_unverified_low_risk and g.risk == "low":
            return _out(GateState.ALLOWED_NOT_VERIFIED, A, "unverified_low_risk")
        return _out(GateState.ALLOWED_NOT_VERIFIED, Q, "no_trusted_check")
    if g.verification is VerificationStatus.FAILED:
        return _out(GateState.BLOCKED, B, "verification_failed")
    if g.verification is VerificationStatus.UNAVAILABLE:
        return _out(GateState.UNVERIFIABLE, Q, "verification_unavailable")
    if g.reconciled_unchanged is None:
        return _out(GateState.BLOCKED, B, "reconciliation_missing")
    if not g.reconciled_unchanged:
        if g.reverify_count >= g.policy.max_reverify:
            return _out(GateState.BLOCKED, B, "verification_not_stable")
        return _out(GateState.REVERIFY, B, "state_changed_during_verification")
    return _out(GateState.ALLOWED_VERIFIED, A, "verified_and_reconciled")
```

harness/gate/decision.py -- a pure function, no I/O. test_all.py drives all ten branches above directly by constructing GateInput fixtures, including confirming a PASSED-but-never-reconciled input is BLOCKED rather than silently treated as verified.

NOTE. Because decide() is pure -- it touches no filesystem, no clock, no process -- it is the cheapest part of the whole harness to test exhaustively. There is no excuse for shipping a gate with an untested branch; the reference test suite exercises every row of the table above directly.

14. Evaluation Layer

Verification (Section 12) asks whether the change is *correct* by some pre-registered, mechanical check. The evaluation layer sits above it and asks a broader, softer question: is the change *good*? This covers three kinds of judgment that a pass/fail trusted check cannot express on its own:

Evaluation	Typical input	Typical output
Tests	Verification evidence for the test-runner check(s)	pass / fail plus which tests, already produced by Section 12
Architecture	Diff shape: files touched, layering rules, import graph	a risk-weighted note, not a pass/fail -- e.g. "touches a module outside the task's declared scope"
Quality	Static analysis, complexity deltas, style	a score or a list of findings feeding the acceptance policy, not the gate directly

The important design constraint is that evaluation output feeds the *acceptance policy* the gate consults -- it never bypasses the gate directly. A high quality score cannot substitute for a failed trusted check, and a low quality score on an otherwise-verified change is a policy decision (block, ask, or accept-with-a-flag), not a hardcoded override.

15. Observability, Evidence & Receipts

Three related but distinct artifacts come out of a run, and it is worth keeping their names and purposes separate:

- **Trace** -- an append-only, hash-chained log of every event (tool calls, permission decisions, verification runs, gate transitions) as they happen. Its job is forensic: reconstruct exactly what occurred, in order, and detect tampering.
- **Evidence** -- a small number of structured, fingerprinted records (permission evidence, verification evidence, evaluation evidence) that the gate actually consumes as typed input, distinct from the free-form trace.
- **Receipt** -- a single signed-by-hash summary of the final decision, referencing the evidence digests, the catalog fingerprint, the command fingerprints, and the workspace fingerprint the decision was about. This is the one artifact a downstream system (CI, a human reviewer) should need to check.

Hash-chained trace

```
class TraceWriter:
    def emit(self, kind: str, data: dict) -> str:
        ev = {"schema": 1, "job_id": self.job_id, "seq": self._seq,
              "ts": now_ts(), "kind": safe_id(kind, "kind"),
              "data": data, "prev": self._prev}
        ev["hash"] = fingerprint(ev, "trace-event/v1")   # computed w/o "hash"
        os.write(self._fd, canonical_json(ev) + b"\n")
        os.fsync(self._fd)                               # durable before use
        self._prev, self._seq = ev["hash"], self._seq + 1
        return ev["hash"]

def verify_chain(path: Path) -> int:
    """Return number of valid events or raise ContractError at first break."""
    prev, n = GENESIS, 0
    for line in path.read_bytes().splitlines():
        ev = json.loads(line)
        claimed = ev.pop("hash", None)
        if ev.get("prev") != prev or ev.get("seq") != n:
            raise ContractError(f"trace broken at seq {n}")
        if fingerprint(ev, "trace-event/v1") != claimed:
            raise ContractError(f"trace tampered at seq {n}")
        prev, n = claimed, n + 1
    return n
```

harness/observability/trace.py -- test_all.py writes two events, confirms the chain verifies, then tampers with the first event's payload in place and confirms verify_chain raises at seq 0.

Receipt

```
def build_receipt(*, job_id, session_id, decision, evidence,
                  catalog_fp, command_fps, workspace_fp, policy_fp,
                  mutation_revision) -> dict:
    body = {"schema_version": 1, "job_id": safe_id(job_id, "job_id"),
            "session_id": safe_id(session_id, "session_id"),
            "decision": decision.value,
            "evidence": sorted(e.digest() for e in evidence),
            "catalog_fp": catalog_fp, "command_fps": sorted(command_fps),
            "workspace_fp": workspace_fp, "policy_fp": policy_fp,
            "mutation_revision": mutation_revision, "issued_at": now_ts()}
    return {**body, "receipt_id": fingerprint(body, "receipt/v1")}

def verify_receipt(receipt: dict, *, current_workspace_fp: str) -> None:
    body = {k: v for k, v in receipt.items() if k != "receipt_id"}
    if fingerprint(body, "receipt/v1") != receipt.get("receipt_id"):
        raise ContractError("receipt was altered")
    if receipt["workspace_fp"] != current_workspace_fp:
        raise ContractError("receipt does not describe the current workspace")
```

harness/evidence/receipt.py -- test_all.py confirms editing any field of a built receipt (including swapping the decision from BLOCKED to ALLOWED) is caught by verify_receipt, and that a receipt is correctly rejected when checked against a different current workspace fingerprint than the one it names.

> **NOTE.** A receipt is only meaningful alongside a live workspace fingerprint check. A perfectly valid, unaltered receipt for yesterday's workspace state says nothing about today's -- always call verify_receipt with the fingerprint of the workspace you are about to trust, not just check that the receipt itself is internally consistent.

16. Contracts & Schemas

Every module in this design imports from a single harness/contracts/core.py with no dependencies beyond the standard library, so the definitions of "safe identifier," "canonical JSON," and "fingerprint" are identical everywhere they are used -- there is exactly one place that could get these wrong.

Safe identifiers and timestamps

```
_SAFE_ID = re.compile(r"[A-Za-z0-9](?:[A-Za-z0-9_-]{0,62}[A-Za-z0-9])?")
_TS = re.compile(r"\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d{1,6})?Z")

def safe_id(value: object, field: str) -> str:
    """IDs become file names and log keys: no dots, slashes or spaces."""
    if not isinstance(value, str) or not _SAFE_ID.fullmatch(value):
        raise ContractError(f"{field}: unsafe identifier")
    return value
```

harness/contracts/core.py -- test_all.py confirms safe_id rejects '../x', embedded spaces, a leading hyphen, an empty string, and a dotted identifier, while accepting ordinary hyphenated IDs.

Exact-keys validation

Schema validation in this design always checks both directions: missing required keys *and* unrecognized extra keys. An extra key that validation silently ignores is exactly the shape of bug that lets a malformed or malicious catalog, policy, or trace event smuggle in a field some other, less careful code path later reads.

```
def exact_keys(obj: object, required, optional=(), where="object") -> dict:
    if not isinstance(obj, dict):
        raise ContractError(f"{where}: expected an object")
    keys, req = set(obj), set(required)
    missing, extra = req - keys, keys - (req | set(optional))
    if missing or extra:
        raise ContractError(
            f"{where}: missing={sorted(missing)} unknown={sorted(extra)}")
    return obj
```

harness/contracts/core.py -- used by harness/verification/catalog.py to validate the catalog document and each check entry.

Canonical JSON and domain-separated fingerprints

Fingerprints throughout this design (context, workspace, catalog, command, trace event, receipt) all go through the same two functions: canonical_json, which sorts keys, forbids floats (non-reproducible across platforms) and forbids unsupported types outright, and fingerprint, which hashes a domain-separation string plus a NUL byte plus the canonical bytes -- so a workspace fingerprint and a catalog fingerprint can never collide just because two different objects happened to canonicalize to the same bytes.

```
def canonical_json(obj: object) -> bytes:
    _check_json(obj)   # rejects floats and unsupported types
    return json.dumps(obj, sort_keys=True, separators=(",", ":"),
                      ensure_ascii=True, allow_nan=False).encode("ascii")

def fingerprint(obj: object, domain: str) -> str:
    """SHA-256 over domain + NUL + canonical JSON (domain separation)."""
    h = hashlib.sha256()
    h.update(domain.encode("ascii") + b"\x00")
    h.update(canonical_json(obj))
    return "sha256:" + h.hexdigest()
```

harness/contracts/core.py -- test_all.py confirms the same payload fingerprinted under two different domain strings produces two different hashes, and that a float anywhere in the payload raises before hashing.

Schema version registry

Schema	Current version	Producer
trace	1	harness/observability/trace.py
report	1	evaluation layer (Section 14)
decision	1	harness/gate/decision.py
policy	1	harness entry point (Section 4)
receipt	1	harness/evidence/receipt.py
catalog	1	harness/verification/catalog.py

Bump the relevant version the moment a schema's required or optional key set changes, and reject documents that declare an unsupported version rather than attempting best-effort forward compatibility -- a harness silently guessing at an unfamiliar schema is worse than one that refuses to start.

17. Security Model

Sections 9-12 already introduced most of the individual defenses; this section collects them as a single threat-model view, because each defense on its own answers "what could go wrong here" and it is easy to miss that they are covering for each other.

17.1 Symlink defense

A workspace is not a closed world: OpenCode's own edit tool, or a malicious dependency pulled in during the run, can plant a symlink that points anywhere on the filesystem the harness process can read or write. Two independent defenses apply:

- **Snapshot / hash time:** every stat is an lstat (follow_symlinks=False), and every file read for hashing opens with O_NOFOLLOW, so a symlink is recorded as a symlink and never dereferenced.
- **Path resolution time:** resolve_inside() is a cheap advisory pre-check that a candidate path, once symlinks are resolved, stays under the workspace root -- useful for early rejection but not sufficient on its own (see 17.2).

```
def resolve_inside(root: Path, candidate: str) -> Path:
    """Cheap pre-check (advisory only): resolves symlinks, must stay in root."""
    root_r = root.resolve(strict=True)
    p = (root_r / candidate).resolve(strict=False)
    if p != root_r and root_r not in p.parents:
        raise PathEscape(f"{candidate!r} escapes the workspace")
    return p
```

harness/workspace/paths.py -- test_all.py confirms a symlink planted inside the workspace pointing at /etc/passwd is correctly rejected by resolve_inside.

17.2 TOCTOU defense

A pre-check like resolve_inside has a gap: the filesystem can change between the check and the later open. The authoritative defense is open_beneath, which walks the path one component at a time from a held directory file descriptor, opening every intermediate component with O_NOFOLLOW so a symlink swapped in after the check -- but before the open -- cannot redirect the operation outside the workspace.

```
def open_beneath(root_fd: int, rel: str, flags: int = os.O_RDONLY) -> int:
    """Authoritative open: walk component by component with O_NOFOLLOW so a
    symlink swapped in after the pre-check cannot redirect us (TOCTOU)."""
    parts = [p for p in rel.split("/") if p not in ("", ".")]
    if ".." in parts:
        raise PathEscape("'..' is not allowed")
    fd = os.dup(root_fd)
    for i, part in enumerate(parts):
        last = i == len(parts) - 1
        mode = flags if last else os.O_RDONLY | os.O_DIRECTORY
        nxt = os.open(part, mode | os.O_NOFOLLOW, dir_fd=fd)
        os.close(fd)
        fd = nxt
    return fd
```

harness/workspace/paths.py -- test_all.py opens a real file through open_beneath and confirms it reads the expected contents, then confirms attempting to open a path component that is itself a symlink raises OSError (ELOOP) rather than following it.

SECURITY. resolve_inside and open_beneath are deliberately two different functions with two different trust levels. Use the cheap one for early UX-level rejection ("that path looks wrong"); use the expensive one as the actual gate immediately before any operation whose result you will trust.

17.3 Process containment

Section 10 covered the mechanics; the security framing is that an untrusted command must never be able to outlive the harness's accounting of it. start_new_session=True puts the child in its own process group so os.killpg can terminate the entire tree -- including children the command itself spawned in the background -- rather than only the direct child subprocess is watching.

17.4 Trust boundaries

Boundary	Trusted side	Untrusted side	Enforced by
Workspace contents	Nothing	Everything under the workspace root, including its own config files	Snapshot hashing, immutable-input pinning (12.1)
Verification executable	Path outside the workspace, root/self-owned ancestry	Any path inside the workspace, any world-writable ancestor	validate_check (12.2)
Catalog document	The single, pre-registered file the entry point was pointed at	Any document an agent could have written or modified	load_catalog ancestry + schema check (12.1)
Tool call target	Nothing by default (default-deny)	Every tool call from OpenCode	Permission controller (9)
Trace / receipt	The hash chain / receipt digest	The raw event payloads and any copy of the file	verify_chain, verify_receipt (15)

18. File & Module Layout

The reference package mirrors the controller boundaries directly: one top-level package per section of this document, no module importing from a sibling it does not conceptually depend on, and harness/contracts/core.py as the only module every other module is allowed to import from freely.

```
harness/
  contracts/    core.py         # Sec 16: IDs, timestamps, schemas, fingerprints
  entry.py                      # Sec 4:  session/job creation, budget, policy load
  context/      select.py       # Sec 5:  budgeted context selection
  models/       route.py        # Sec 6:  profile routing
  orchestration/state.py        # Sec 7:  RunMachine, Budget
  permission/   policy.py       # Sec 9:  risk classification, rule evaluation
  execution/    process.py      # Sec 10: contained subprocess execution
  workspace/    paths.py        # Sec 17.1-17.2: symlink/TOCTOU-safe path ops
                snapshot.py     # Sec 11: before/after snapshot, diff, fingerprint
  verification/ catalog.py      # Sec 12.1-12.2: trusted catalog + identity checks
                select.py       # Sec 12.3: scope/cost based check selection
                runtime.py      # Sec 12.4: run checks, aggregate status
  gate/         decision.py     # Sec 13.1, 13.3: the pure gate function
                reconcile.py    # Sec 13.2: post-verification reconciliation
  observability/trace.py        # Sec 15: hash-chained append-only trace
  evidence/     receipt.py      # Sec 15: Evidence, build_receipt, verify_receipt
```

Directory layout used by the reference package accompanying this document; every leaf module above has a corresponding block in test_all.py or test_more.py.

NOTE. harness/gate/decision.py is the one module in this layout with zero imports outside harness.contracts. Keeping the gate free of filesystem, process, and network access is what makes Section 13.3's exhaustive branch testing possible in the first place -- protect that property as the codebase grows.

19. Testing Strategy

The reference package was built test-first against real processes and a real filesystem rather than mocks, because most of what this harness exists to prevent -- a symlink escape, a TOCTOU race, a stray process, a tampered trace -- only shows up when the underlying primitive actually behaves like the operating system, not like a mock's idea of it.

Layer	What was actually exercised
Contracts	Identifier and timestamp regexes against valid and invalid strings; canonical JSON rejecting floats; fingerprint domain separation; exact_keys against missing and extra keys
Snapshot / paths	A real directory tree mutated between two snapshots (add, modify, delete, symlink-plant); open_beneath against a real O_NOFOLLOW failure; resolve_inside against a real symlink escape
Execution	A real short-lived process, a real timeout against a 30-second sleep, a real background child leaked by a shell wrapper confirmed killed (verified via /proc state), and a rejected relative-path argv[0]
Verification	A catalog loaded from a world-writable path (rejected), a catalog inside the workspace (rejected), an immutable input tampered after pinning (rejected), an executable inside the workspace (rejected), and six real subprocess outcomes mapped through run_check / aggregate
Gate	All ten rows of the decision table in Section 13.3 driven directly as GateInput fixtures
Observability / evidence	A two-event trace chain verified, then tampered and confirmed to fail at the correct sequence number; a receipt tampered in three different fields and confirmed rejected each time

NOTE. None of the tests above use unittest.mock for the security-relevant behavior. A mocked os.symlink or a mocked subprocess.Popen proves the code calls the right function; it does not prove the function does the right thing when the kernel is actually involved.

20. Failure Modes & Operational Notes

Symptom	Likely cause	Where to look
Every run ends ASK_USER even for trivial edits	No catalog check scoped to the paths being touched	Section 12.3 -- widen scope globs or add a low-cost check
Runs intermittently loop through REVERIFY and then BLOCK	A check has a side effect that is not idempotent (writes a timestamped file, appends to a log inside the workspace)	Section 13.2 -- make the check's own writes land outside the workspace, or add its output path to an ignore list
A previously-passing catalog suddenly fails to load	catalog file or an ancestor directory's permissions changed (became group/world writable)	Section 12.1 -- assert_trusted_ancestry; check `ls -ld` up the path
Harness process accumulates zombie children over many runs	The container/init is not reaping; run_contained kills but relies on the surrounding process supervisor to reap	Section 10, 17.3 -- confirm PID 1 in the execution environment reaps zombies (most container runtimes do this automatically)
verify_chain fails on an old trace file after a schema bump	SCHEMA_VERSIONS was incremented without a migration path	Section 16 -- keep old trace readers around, or run a one-time migration before verifying archives

Appendix A: Verified Code Listings

All code in the body of this document is reproduced from the modules below, each of which was written and exercised with real filesystem and process operations while this document was produced. This appendix lists every module and its companion test file for reference; the full listings are not repeated here since each one already appears in the section that discusses it.

Module	Discussed in	Exercised by
harness/contracts/core.py	Section 16	test_all.py
harness/workspace/paths.py	Section 17.1-17.2	test_all.py
harness/workspace/snapshot.py	Section 11	test_all.py
harness/execution/process.py	Section 10	test_all.py
harness/verification/catalog.py	Section 12.1-12.2	test_all.py
harness/verification/select.py	Section 12.3	test_all.py

Module	Discussed in	Exercised by
harness/verification/runtime.py	Section 12.4	test_more.py
harness/gate/decision.py	Section 13.1, 13.3	test_all.py
harness/gate/reconcile.py	Section 13.2	(exercised via snapshot/diff primitives already tested in test_all.py)
harness/orchestration/state.py	Section 7	test_all.py
harness/permission/policy.py	Section 9	test_all.py
harness/observability/trace.py	Section 15	test_all.py
harness/evidence/receipt.py	Section 15	test_all.py
harness/context/select.py	Section 5	test_all.py
harness/models/route.py	Section 6	test_all.py

Appendix B: Glossary

Term	Definition
Catalog	The trusted, out-of-workspace document listing every verification check the harness may run.
Evidence	A structured, fingerprinted record of one fact the gate consumes (a permission decision, a verification result, an evaluation).
Fingerprint	A domain-separated SHA-256 hash of a canonical JSON encoding of some object, used to name a piece of state without ambiguity.
Gate	The pure decision function that turns evidence plus policy into ALLOWED / BLOCKED / ASK_USER.
Mutation revision	A monotonically increasing counter bumped every time a snapshot diff shows a change, including changes made by verification itself.
Receipt	The single hash-signed summary document of a run's final decision and the evidence behind it.
Reconciliation	Re-diffing the workspace after verification to confirm the checks did not themselves mutate the state being judged.
Trace	The append-only, hash-chained log of every event in a run, used for forensic reconstruction.
TOCTOU	Time-of-check to time-of-use: a race between validating a path and later using it, during which the filesystem can change.
Trusted ancestry	The property that a file and every directory above it up to the filesystem root is owned by root or the harness's own user and not group/world writable.
Verification	Running an independent, pre-registered check against a workspace change and reporting PASSED / FAILED / UNAVAILABLE.